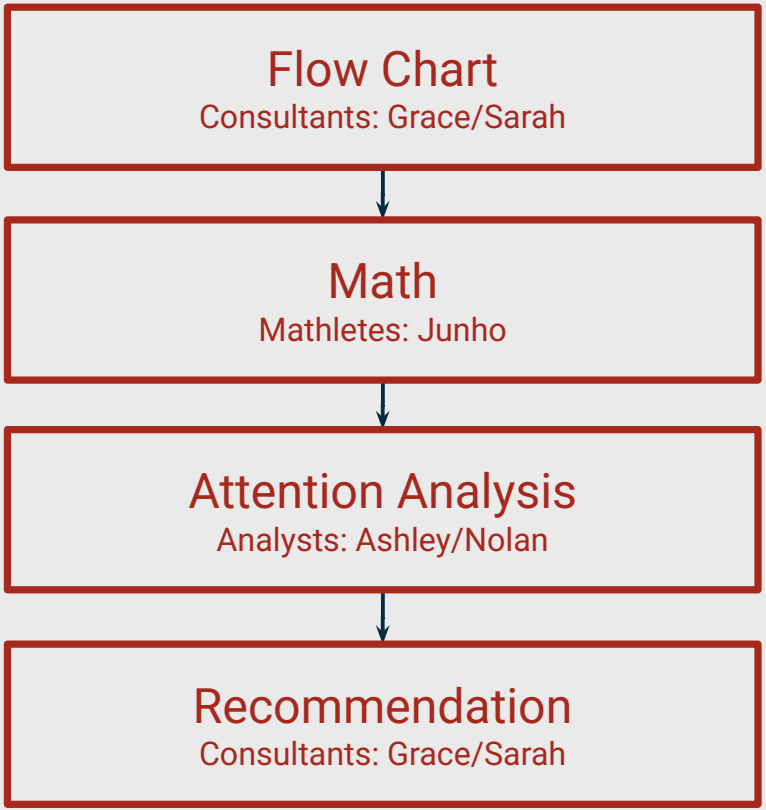
Ashley Cheng, Grace Fan, Junho Hong, Sarah Villamil, Nolan White

Presentation 1

Team1_FullStack

Ashley Cheng, Grace Fan, Junho Hong, Sarah Villamil, Nolan White

Overview

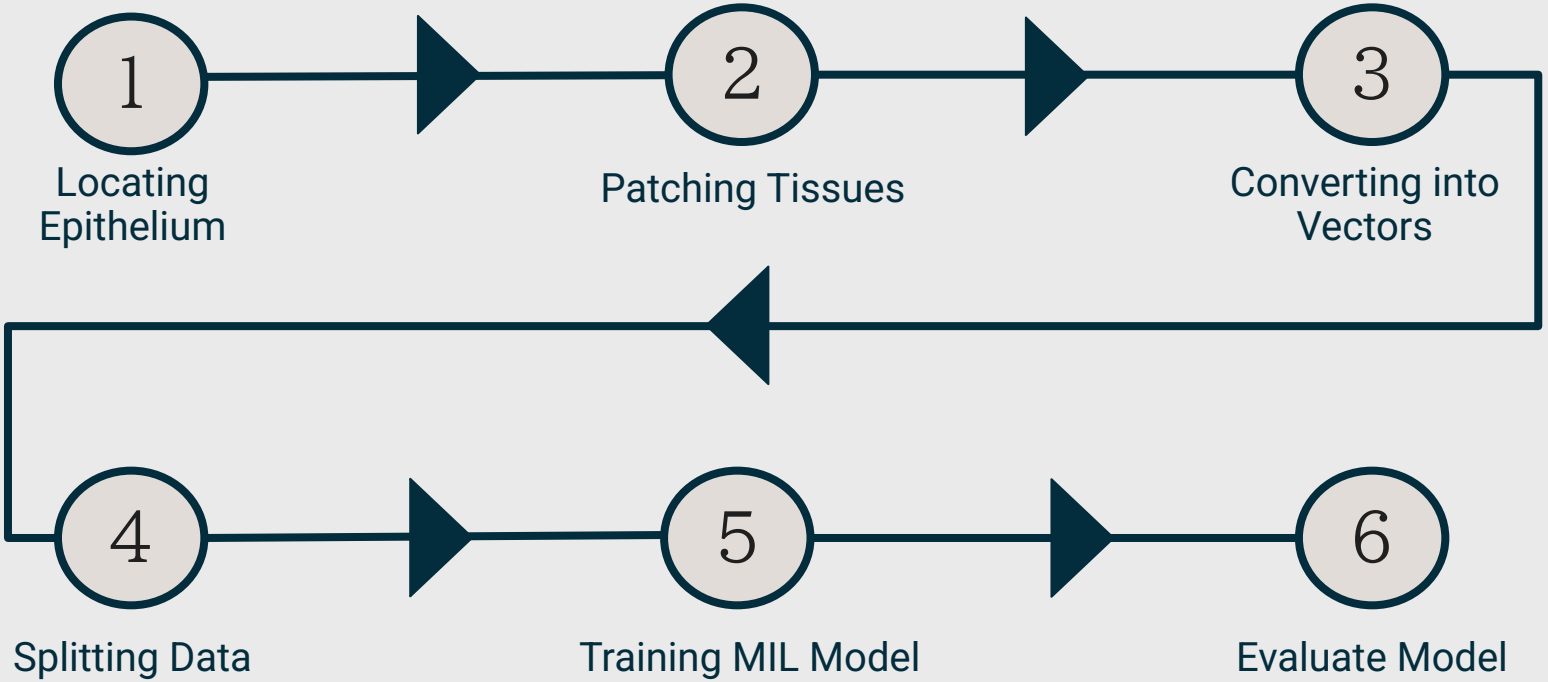


Grace/Sarah

Flowchart

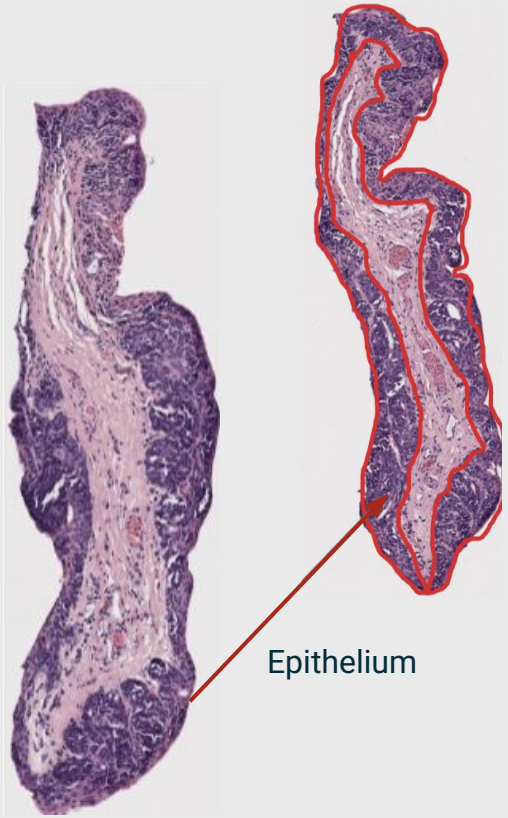
Consultants

Workflow Overview



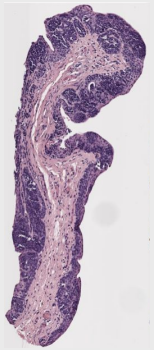
Step 1: Locating Epithelium (Stage 1)

- Using pathologist annotations and Labeled Image Server
- Locate dark purple boundary as the epithelium for analysis
- Data engineers



Given Circumstances

- 105 cases from 97 patients
- Each case has multiple slices of tissue
- 3 Stains:



IHC with H&E

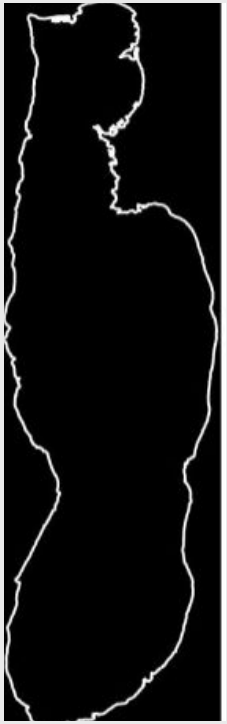


IHC with Melan-A



IHC with SOX10

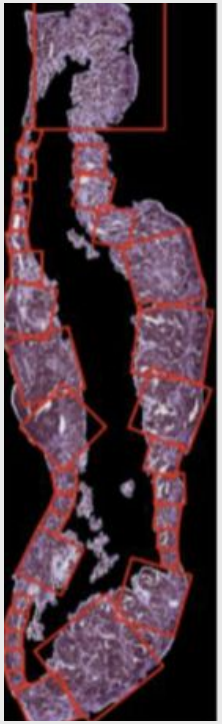
Step 2: Patching Tissues (Stage 1)



Contour of Slice



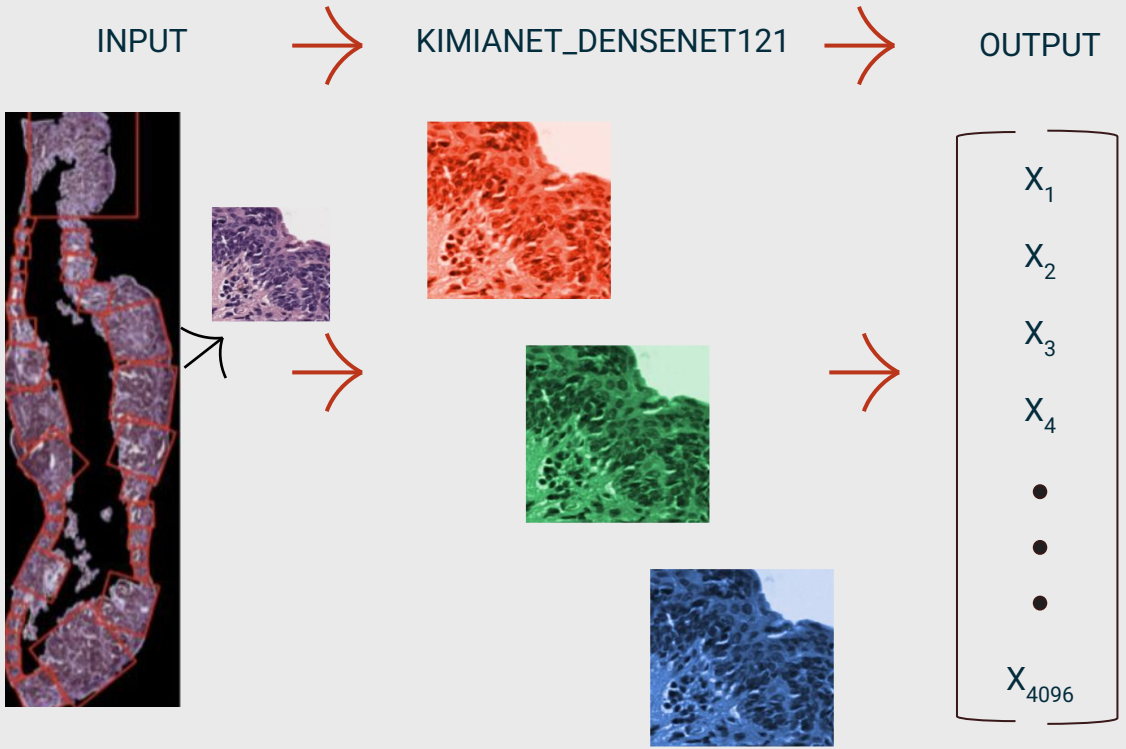
Normal Vector



Patches around the epithelium

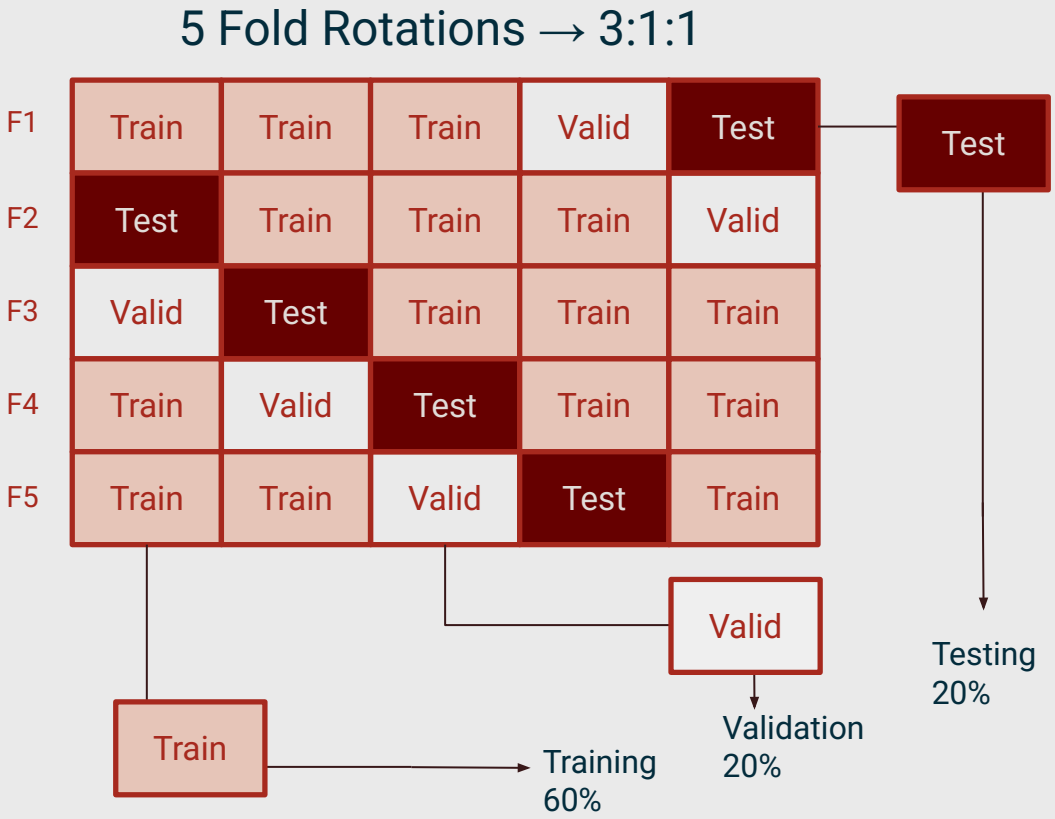
- Each Slice of Tissue is patched
- Using the outer border of epithelium to find contour
- Use thickness of epithelium and Normal vectors
- Create square patches

Step 3: Converting into Vectors (Stage 2)



- Uses KimiaNet encoder: densenet121
- Previously trained model, frozen and used to detect features
- Input patches as RGBs
- Output 4096 dimensional feature vectors (edges, textures, patterns, etc..)

Step 4: Splitting Data (Stage 3)



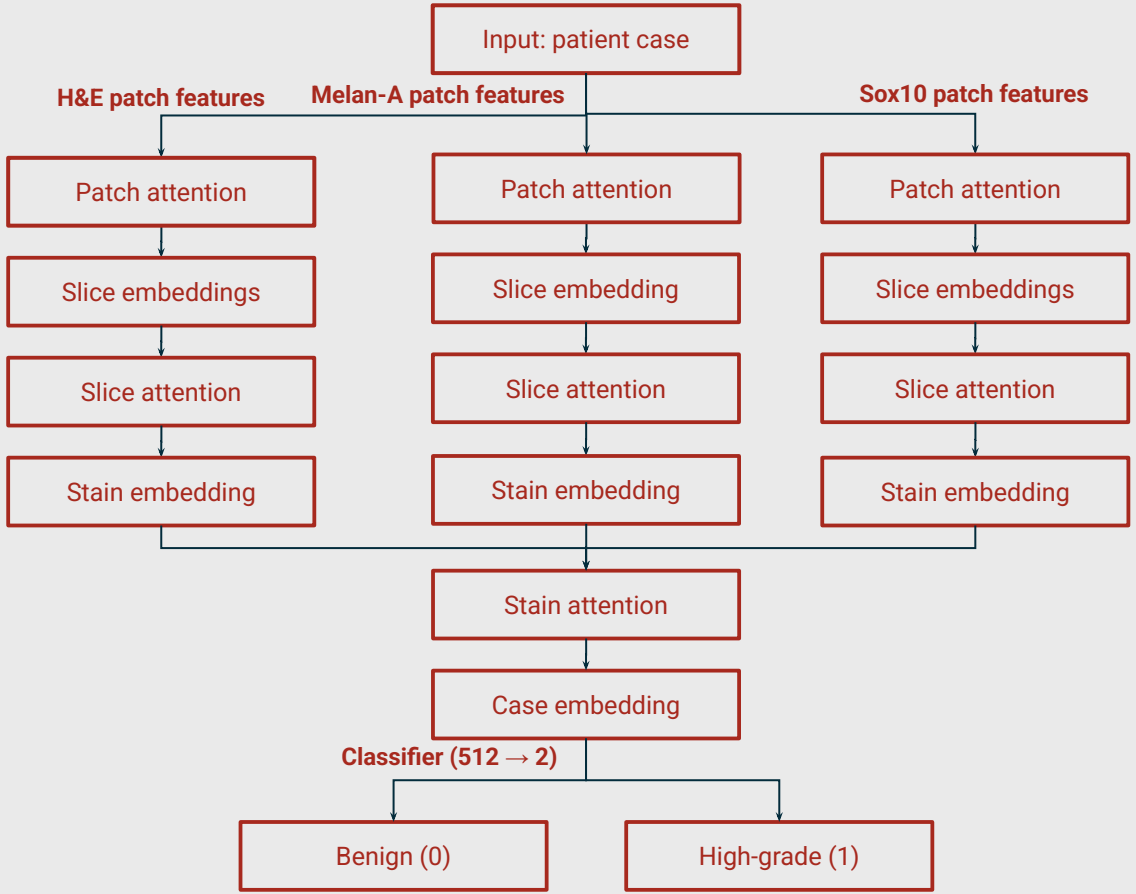
- Split data into

  Train: 60%

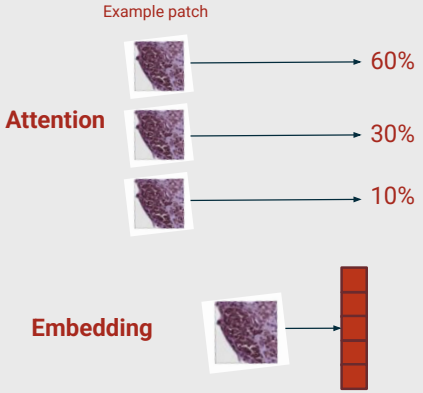
  Validation: 20%

  Test: 20%
- Five folds
- Stratified
- leakage-free by grouping cases of the same patients together

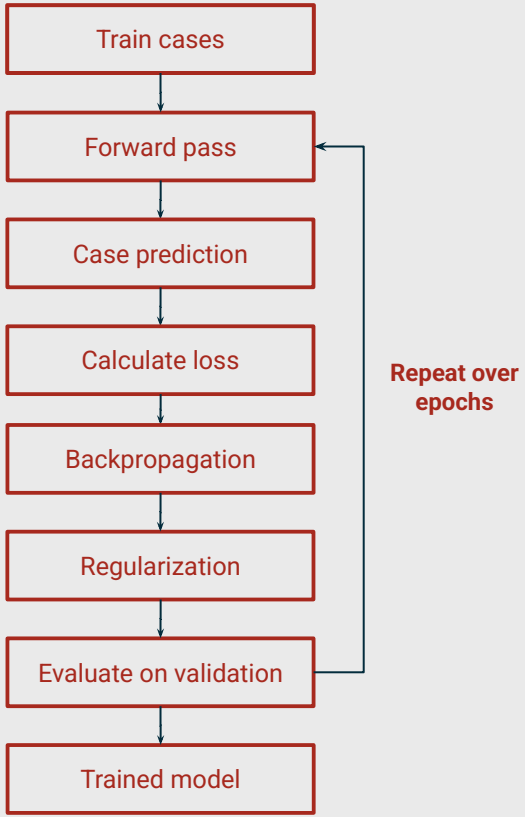
Step 5: MIL Model Architecture (Stage 4)



- **Case → prediction**
- **Combines information from patches → slices → stains → case**
- **Uses attentions at each level**

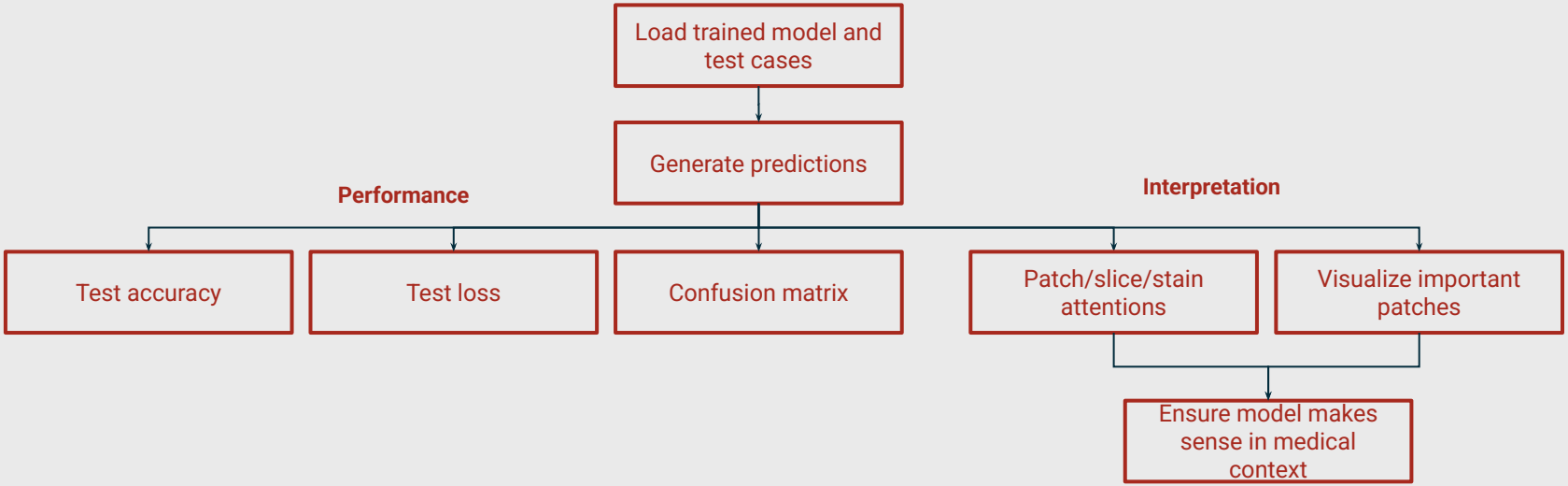


Step 5: Model Training (Stage 4)



- **Learns attention weights**
- **Compares predictions to true labels and adjusts parameters**

Step 6: Evaluate Model (Stage 4)



- **Learns attention weights**
- **Compares predictions to true labels and adjusts parameters**

Junho

Math

Mathletes

Four issues hide in the pipeline

Data and splits
93 cases, 5 folds
1

Patch features
KimiaNet, frozen

Attention MIL
patch, slice, stain

Loss
CE − λ·entropy
2 3

Training
AdamW, Optuna
3 4

1 tuning leak 2 class weight 3 inactive regularisers 4 last-epoch test

The whole model in one line

$$p_c = \mathrm{softmax}\left(W_c \cdot Attn_{stain} \circ Attn_{slice} \circ Attn_{patch}\left(\mathrm{ReLU}(X W_p^{\top} + b_p) \right) + b_c \right)$$

X: patch features of a slice · W_p, b_p: projector · Attn: gated-attention pooling · W_c, b_c: classifier · p_c: class probabilities of case c

Base code: Presentation1_base_code (make_splits.py, models.py, trainer.py, main.py, optuna_training.py)

Validation folds cover every case

Labels (classes 1, 3, 4 kept: 93 cases)

$$y_c = 1[\,Class_c \in \{3, 4\}\,]$$

Patch filter (2.4% of patches removed)

$$P_{c,s,t} = \{\, x : min(h_x, w_x) \geq 32 \,\}$$

Patient grouping

$$g(c) = g(c') \;\Rightarrow\; fold(c) = fold(c')$$

y_c: label (1 = high-grade) · $P_{c,s,t}$: patches of slice t, stain s
h_x, w_x: patch height, width · g(c): patient of case c · fold(c): its fold

Folds F1–F5 rotate through train, val and test

	F1	F2	F3	F4	F5
Split 1	train	train	train	**val**	test
Split 2	test	train	train	train	**val**
Split 3	**val**	test	train	train	train
Split 4	train	**val**	test	train	train
Split 5	train	train	**val**	test	train

1 Hyperparameters were tuned on the test cases

config.py uses values from an Optuna study that minimises mean validation loss over all 5 splits. Those validation folds cover every case, so test cases shaped the hyperparameters and the reported accuracy is optimistic. Fix: nested cross-validation (Varma & Simon, 2006).

Base code: data_utils.py L114–116 · make_splits.py L149–166, L229–302 · optuna_training.py L381–420, L550–560
My code: https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation1/Code1_PatchSizeFilter_22Oct_Junho.py

2.5 M weights learned from about 56 cases

$$X \in \mathbb{R}^{P\times 4096} \to H \in \mathbb{R}^{P\times 512} \to z_{s,t} \to z_s \to z_c \in \mathbb{R}^{512} \to p \in \mathbb{R}^2$$

projector · patch attention · slice attention · stain attention · classifier

P: patches per slice (≤ 500) · $z_{s,t}$, z_s, z_c: slice, stain and case vectors · p: class probabilities

Each attention level scores every instance, applies a softmax, and takes the weighted average

$$a_i = \mathrm{softmax}_i\big(w^{\top}[\, \tanh(V h_i + b_V) \odot \mathrm{sigm}(U h_i + b_U) \,] \big), \qquad z = \Sigma_i\, a_i\, h_i$$

h_i: instance vector · a_i: its weight · V, U, b_V, b_U, w: learned · ⊙: elementwise product · z: pooled vector

2,493,061

trainable parameters (KimiaNet frozen) ≈ 44,500 per training case

Base code: models.py L8–48 (GatedAttentionPool), L51–235 (HierarchicalAttnMIL)

My code: https://github.com/arvindkrishna87/Team1_FullStack/blob/main/Presentation1/Code2_MathChecks_22Oct_Junho.py

The loss's two safeguards are inactive

$$L_c = w(y) \cdot [\, -\log p(y \mid c) \,] \;/\; w(y) \;-\; \lambda \cdot H_c$$

weighted cross-entropy — attention-entropy bonus

w(y): class weight (benign 2.5285, high-grade 1) · p(y | c): predicted probability of the true class
λ: entropy weight (0.001) · H_c: mean normalised patch-attention entropy, from 0 to 1

2 The class weight cancels

PyTorch divides by the weight sum of the batch. At batch size 1, w(y)/w(y) = 1, so the benign weight never acts. Fix: reduction='sum' or oversample benign (Buda et al., 2018).

3 The entropy bonus is too weak to act

$H \in [0, 1]$ and λ = 0.001, so the bonus is at most 0.001. AEM, the method this follows, uses λ = 0.005–0.1 on unnormalised entropy: 30–600× stronger at 500 patches.

Base code: trainer.py L69 (weighted CE), L159–205 (entropy), L264–268 (loss) · PyTorch CrossEntropyLoss docs · Zhang et al., AEM

Decay is near zero and the best model goes unused

AdamW step

$$\theta \leftarrow \theta - \eta \left(m / \sqrt{v} + \lambda_{wd} \theta \right)$$

Shrink per step

$$\eta \cdot \lambda_{wd} = 2.08 \cdot 10^{-4} \times 3.05 \cdot 10^{-4} \approx 6 \cdot 10^{-8}$$

θ: model weights · η: learning rate · λ_{wd}: weight decay
m, v: Adam's running averages of gradient and squared gradient

Validation loss by epoch (illustration)



3 Weight decay is effectively zero

Weights shrink about 0.01% over a whole run (~1,700 steps). AdamW multiplies the decay by η, so λ_{wd} must be much larger (Loshchilov & Hutter, 2019).

4 The test uses the last epoch

Early stopping saves best.pth, but main.py evaluates the final weights, 8 epochs past the best whenever early stopping fires. Fix: load best.pth before evaluate().

Base code: trainer.py L57–61 (AdamW), L370–378 (best.pth), L477–494 (early stopping) · main.py L289–305 (evaluate)

Fix four things before chasing 95%

	Issue	Fix
1	Tuning saw the test cases	**Nested cross-validation**
2	Class weight cancels at batch size 1	**reduction='sum' or oversample benign**
3	Entropy bonus and weight decay inactive	**Rescale λ and λ_{wd}, then re-tune**
4	Test runs on the last epoch	**Load best.pth before testing**

Then measure 95% properly

A test fold has 18–19 cases: one error moves accuracy 5.3 points, and 18/19 = 94.7%. Pooled over all 93 cases, cross-validated accuracy still carries roughly ±10% (Varoquaux, 2018). Report pooled accuracy with a confidence interval, plus sensitivity and specificity.

References: Varma & Simon 2006 · Zhang et al. (AEM) · Loshchilov & Hutter 2019 · Varoquaux 2018 · derivations: Mathletes_Writeup_Presentation1_Junho.md

Ashley/Nolan

Attention Analysis

Analysts

1. Attention_analysis Walkthrough: Overview

High-level overview:

- Asks the model to **show its work**
- Points to the evidence the model used to make its decision by showing which patches the model weighted most heavily
- Returns the tissue image for **human validation**
    - Ensures model reasoning is **based on pathology** and valid for medical context

How it works:

Identify

- Evaluates attention at **3 levels of hierarchy**:
    - Stain, slice, patch weights
- Weights sum to 1 in each level → how influential each patch/slice/stain is to its slice/stain/case
- Effective score = stain weight * slice weight * patch weight = each patch's share of case decision

Visualize

- Map each ranked patch's position back to its file, then to the .png with the same name
- Save labeled image grids with stain, slice, patch and score → enable human review

Code reference: /projects/e32998/STAT390_Fall2026/Team6_InstructionalTeam/Code/Presentation1_base_code/attention_analysis.py

1. Attention_analysis Walkthrough: Flowchart

Repeated for every test case

1. Run model, get weights
analyze_attention_weights

Loops over test cases. Gets the prediction and stain / slice / patch weights. Records the true vs. predicted label.

1b. Side view: one slice

Per stain, plots top/bottom 5 patches inside the most and least-trusted slice.

2. Score every patch
compute_effective_patch_attention

Multiplies stain * slice * patch weight = each patch's share of the case decision. Comparable case-wide.

3. Rank: top and bottom 5
visualize_case_effective_patches

Sorts every patch in the case by its effective score. Keeps the 5 highest and lowest (what it ignored).

4. Find each patch's image
_lookup_png_for_patch
_build_pt_index_for_case_stain

Rebuilds the .pt file order to find the patch's file, then swaps to the .png file with the same name.

4b. If PNG not found

Tries indexing the PNG folder by position. If fails, draws the 4096 features as a heatmap.

5. Save labeled image grids
visualize_case_effective_patches

Titles each patch with stain, slice, patch and score. Saves top_ and bottom_ grids to case_effective_patches.

After all cases

6. Stain attention summary
save_attention_summary

Writes each case's stain weights and most-attended stain to attention summary

7. Distribution plots
plot_effective_patch_attention_distribution_per_case
plot_slice_attention_distribution_per_caseandstain

Per case: histogram of effective scores, and slice weights by rank.

8. Top 5% of patches → CSV
analyze_top_effective_patches_per_case

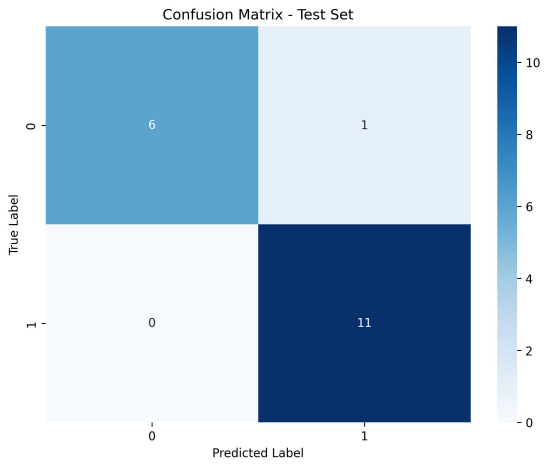
Keeps each case's top 5% of patches, counts their stains and slices, and writes them with true/pred labels.

Code reference: /projects/e32998/STAT390_Fall2026/Team6_InstructionalTeam/Code/Presentation1_base_code/attention_analysis.py

2. Model Results & Visuals

Split One Test Results

Number of Samples: 18
Test Loss: 0.1970
Test Accuracy: 94.44%



* Using split 1. As evidenced by the above confusion matrix, this test split had no false negatives.

Most Confident Cases Are Selected Cases for Visualization:

Category	Count in Category	Most Confident Case	Confidence
Correct High-Grade	11	#5	p=1.00
Correct Benign	6	#119	p=1.00
Incorrect High-Grade	0	None	None
Incorrect Benign	1	#56	p=0.914

Case #56 is the only incorrect classification among the split and has quite high confidence for an incorrect classification.

Run results located at: /projects/e32998/MIL_training/final_runs/run_20261022_115349/

2. Correctly Classified High-Grade: Case 5

Case Level

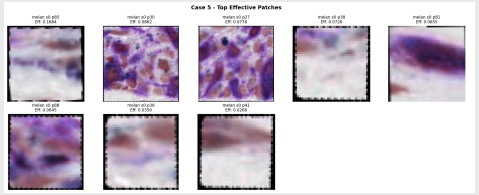
Stain Level

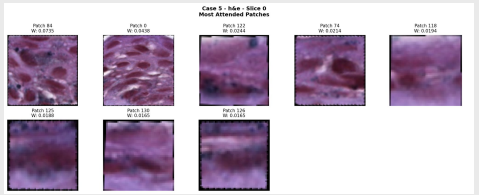
H&E

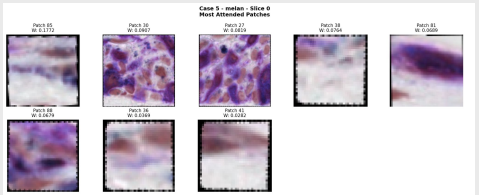
Melan

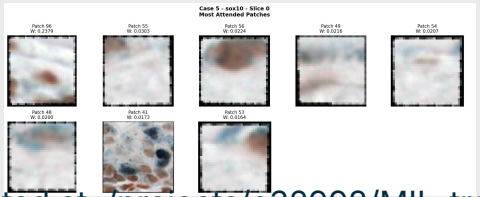
Sox10

Top Effective Patches

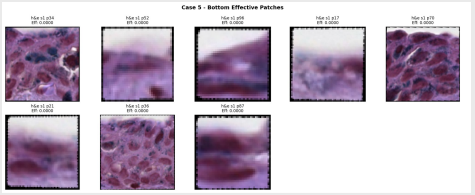


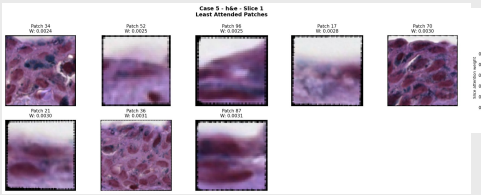


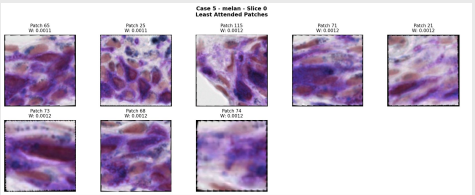


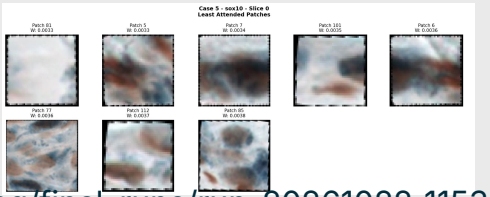


Bottom Effective Patches

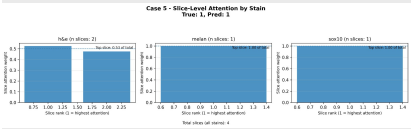








Slice Level



Run results located at: /projects/e32998/MIL_training/final_runs/run_20261022_115349/

2. Correctly Classified Benign: Case 119

Case Level

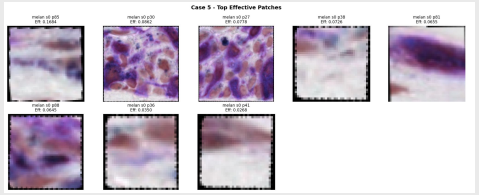
Stain Level

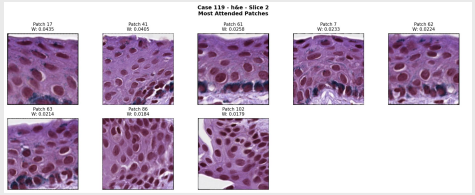
H&E

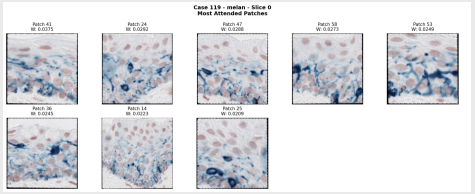
Melan

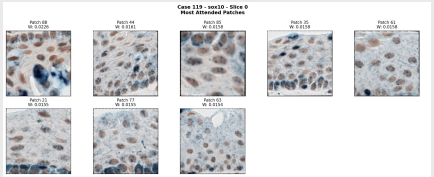
Sox10

Top Effective Patches

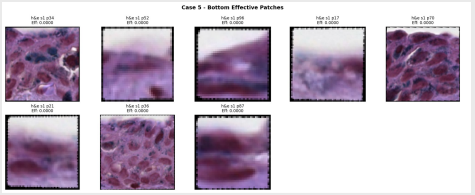


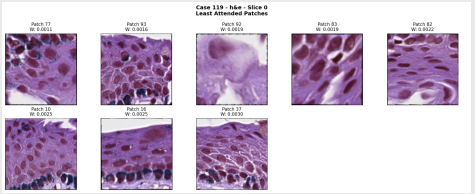


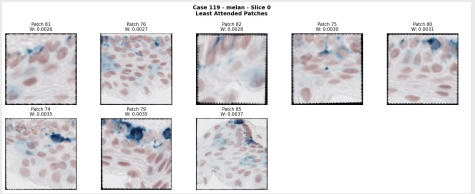


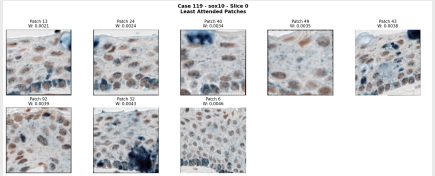


Bottom Effective Patches

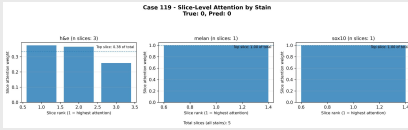








Slice Level



Run results located at: /projects/e32998/MIL_training/final_runs/run_20261022_115349/

2. Incorrectly Classified Benign: Case 56

Case Level

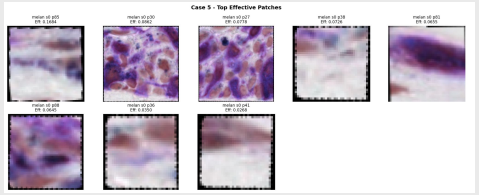
Stain Level

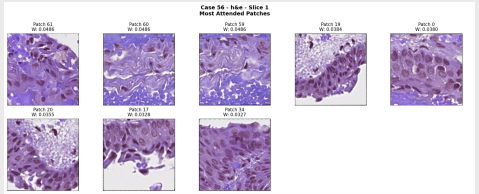
H&E

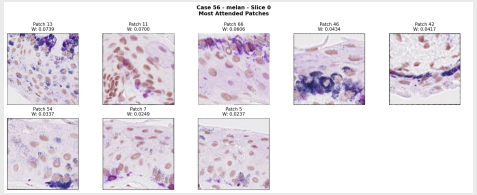
Melan

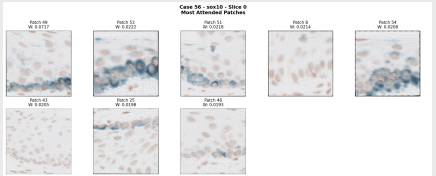
Sox10

Top Effective Patches

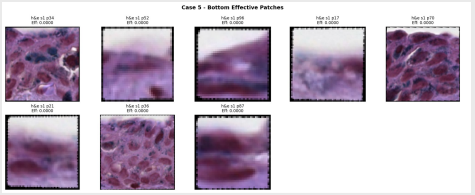


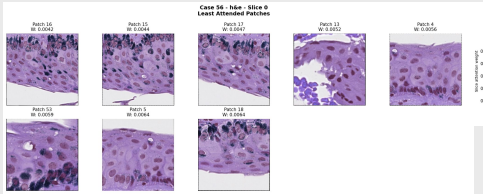


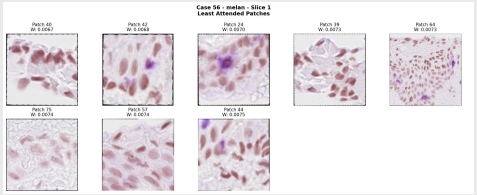


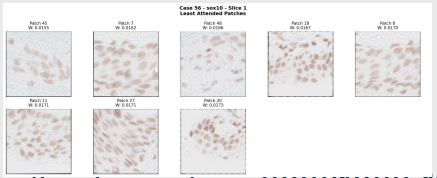


Bottom Effective Patches









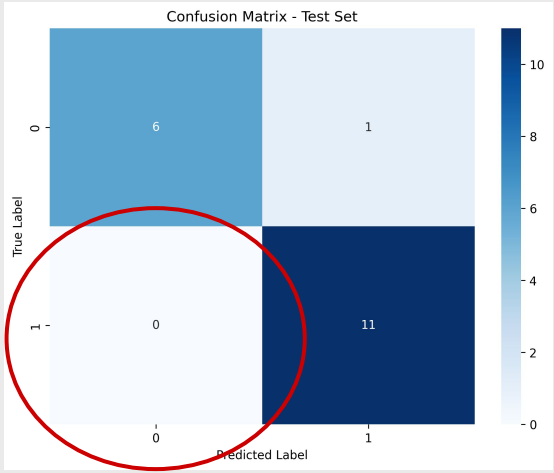
Slice Level



Run results located at: /projects/e32998/MIL_training/final_runs/run_20261022_115349/

Incorrectly Classified High-Grade: None in Split 1

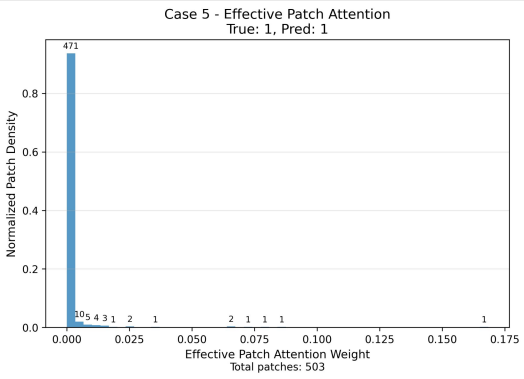
- Among the 18 cases in split 1, no high grade cases classified incorrectly
- Could look at one of the other flour splits to examine this sort of model inaccuracy
- False negatives are highly undesirable in a medical context, as dismissing a malignant case as benign has significant consequences
- Likely a good thing that there are no false negatives here



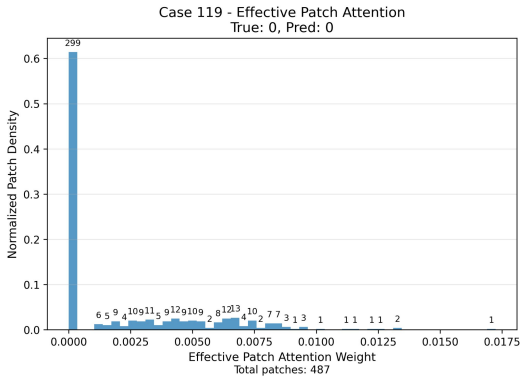
Run results located at: /projects/e32998/MIL_training/final_runs/run_20261022_115349/

Effective Patch Attention Distribution

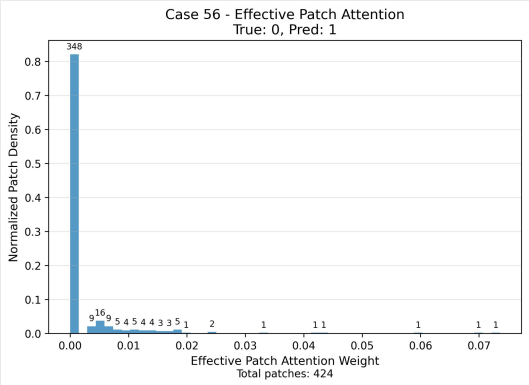
Case 5: Correct High-Grade



Case 119: Correct Benign



Case 56: Incorrect Benign



- The effective patch attention densities are very right skewed
- These graphs show that regardless of true label and predicted label, attention is concentrated in a small number of patches
- This concentration is especially desirable for high-grade cases, since cancer can be localized to a small area in a sample

Run results located at: /projects/e32998/MIL_training/final_runs/run_20261022_115349/

3. Critical Reflection: Does Our Approach Work Well?

Done well:

- Explains the model using the model's actual weights (no extra training or approximation).
- Effective score fits the model hierarchy and pathology.
- A contrast set (bottom_entries) is kept to compare with top patches and determine attention selectiveness.
    - Attention seems relatively concentrated across a small number of patches.

Potential for improvement:

1. There are two distinct indexing conventions, and the code assumes they match up.
    a. Attention Weights: row p of the slice tensor in model
    b. Lookup: p-th file after a resort
    c. For example, we try to load slices 0, 1, and 2. If slice 1 fails to load or is empty, slice 2 gets relabeled as index 1. Every subsequent label is misrepresented

Result: attention weights are correct, but the image assigned to a specific patch might be incorrect.

2. Attention has no direction. A patch with high attention could as easily signal a malignant case as a benign case
    a. Could implement a directional attribution/effective attention score to capture what direction the influence is leaning towards

Code citation: Git hub link: dataset.py, around line 100. Second reference in attention_analysis.py, around line ~300

Recommendation

Consultants

UNI and UNI2

UNI

- Purpose: develop a general-purpose pathology foundation model
- Published March 19, 2024
- Developed by Faisal Mahmood's research group (associated with Harvard Medical School, Brigham and Women's Hospital, Massachusetts General Hospital, and other institutions in Boston)

UNI2

- Purpose: improve the generalizability and quality of pathology images
- Released January 14, 2025
- Developed by same group

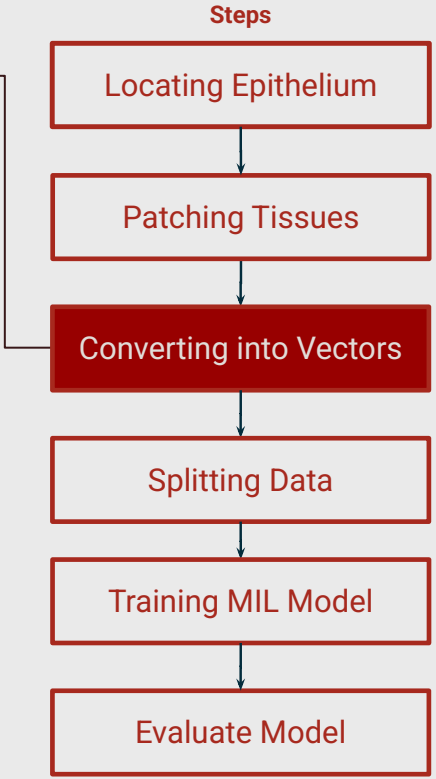
UNI: https://www.nature.com/articles/s41591-024-02857-3
UNI2: https://github.com/mahmoodlab/UNI

KimiaNet vs. UNI/UNI2

Feature	KimiaNet	UNI / UNI2
Model generation	Older (September 2020)	Newer (last two years)
Architecture	DenseNet121 (CNN)	Vision Transformer (ViT)
Training data	The Cancer Genome Atlas (TCGA)	Larger, more diverse pathology dataset
Number of image patches	240,000+	100 million + / 200 million +
Number of Whole-Slide Images	7,126	100,000+ / 350,000+
Tissue types	30 TCGA cancer categories	20 major tissue types / diverse tissues and diseases
Training method	Weakly supervised classification	Self-supervised learning

Implementation

- Apply changes to Step 3: Converting into Vectors (Stage 2)
- Recommended to use UNI2 initially as is most up to date model
  - UNI should also be tried to compare results
- Precompute_pooled_features.py
  - Replace current functions load_kimianet_densenet121()
- UNI/UNI2 takes the same input: patches of tissue
- UNI/UNI2 gives same output: embedding of feature vectors
- Documentation below shows exact steps on how to code and run the model



UNI: https://www.nature.com/articles/s41591-024-02857-3
UNI2: https://github.com/mahmoodlab/UNI